

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO**

**ANDRÉ FABRICIO WOZNIACK
RICARDO HESPANHOL PAMPLONA**

**DETECÇÃO DE VULNERABILIDADES EM CÓDIGO-FONTE COM
MACHINE LEARNING**

PLANO DE PROJETO – 1º BIMESTRE

Orientador: Prof. Julio Cezar Nievola

Visto do Orientador

**CURITIBA
2026**

RESUMO

Palavras-chave: Palavra 1. Palavra 2. Palavra 3.

SUMÁRIO

1	INTRODUÇÃO	3
1.1	MOTIVAÇÃO E RELEVÂNCIA	3
1.2	OBJETIVOS	3
1.2.1	Objetivo Geral	3
1.2.2	Objetivos Específicos	3
1.3	ESTRUTURA DO DOCUMENTO	3
2	DETALHAMENTO DO PROBLEMA	4
3	ESTADO DA ARTE	5
4	TRABALHO A SER DESENVOLVIDO	6
4.1	DESCRIÇÃO FUNCIONAL E DIAGRAMA EM BLOCOS	6
4.2	DETALHAMENTO DOS MÓDULOS FUNCIONAIS	6
5	TECNOLOGIAS QUE SERÃO UTILIZADAS	7
5.1	TECNOLOGIAS ESCOLHIDAS E JUSTIFICATIVAS	7
5.2	TECNOLOGIAS ALTERNATIVAS AVALIADAS	7
6	PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO	8
6.1	PROCEDIMENTOS DE TESTE	8
6.2	CRITÉRIOS DE ACEITE E DESEMPENHO	8
7	ANÁLISE DOS RISCOS	9
8	CRONOGRAMA DO PROJETO	10
9	CONCLUSÃO	11
	REFERÊNCIAS	12

1 INTRODUÇÃO

1.1 MOTIVAÇÃO E RELEVÂNCIA

1.2 OBJETIVOS

1.2.1 Objetivo Geral

1.2.2 Objetivos Específicos

1.3 ESTRUTURA DO DOCUMENTO

2 DETALHAMENTO DO PROBLEMA

Primeiramente para entender os problemas de segurança em código, precisamos definir a produção de código antes das ferramentas de Inteligência Artificial. O código é produzido por desenvolvedores humanos, que escrevem manualmente cada linha de código, utilizando suas habilidades e conhecimentos para criar software funcional. Esse processo envolve a compreensão dos requisitos do sistema, a implementação de algoritmos e estruturas de dados, e a realização de testes para garantir que o código funcione corretamente. No entanto, mesmo com desenvolvedores experientes, erros podem ocorrer, resultando em vulnerabilidades de segurança.

No entanto, estamos vivendo uma verdadeira revolução na produção de código com o advento de ferramentas de Inteligência Artificial, como Claude Code, Codex, Antigravity, Cursor e o GitHub Copilot, que utilizam modelos de linguagem avançados para auxiliar os desenvolvedores na escrita de código. Essas ferramentas podem gerar código automaticamente, sejam por trechos ou até mesmo abordagens como o uso de agentes para criar tudo através com base em descrições em linguagem natural. Embora essas ferramentas possam aumentar a produtividade e reduzir o tempo de desenvolvimento, elas também introduzem desafios de segurança.

Com esse advento, surgem preocupações sobre a qualidade e segurança do código gerado automaticamente. Estudos recentes indicam que código assistido por IA apresenta, em média, 4,4 vezes mais vulnerabilidades que código escrito apenas por humanos, com certos tipos de falha aparecendo com maior frequência e templates inseguros se repetindo em diferentes repositórios, sugerindo padrões induzidos pelos próprios modelos (State of Cyber; Wu *et al.*, 2026, 2025). Embora ferramentas de análise estática ajudem a identificar esses problemas, elas frequentemente geram falsos positivos e possuem dificuldade para compreender o contexto do código, como a intenção do desenvolvedor e chamadas entre funções (Yang *et al.*; TechScience, 2025, 2025). Por isso, há demanda por abordagens mais contextuais, baseadas em Machine Learning e Large Language Models, que vêm sendo aplicadas à detecção de vulnerabilidades e à priorização de alertas em código gerado automaticamente (Wang *et al.*; MDPI, 2025, 2026).

3 ESTADO DA ARTE

4 TRABALHO A SER DESENVOLVIDO

Objetivo : DESENVOLVER um software que analise...

4.1 DESCRIÇÃO FUNCIONAL E DIAGRAMA EM BLOCOS

4.2 DETALHAMENTO DOS MÓDULOS FUNCIONAIS

5 TECNOLOGIAS QUE SERÃO UTILIZADAS

5.1 TECNOLOGIAS ESCOLHIDAS E JUSTIFICATIVAS

5.2 TECNOLOGIAS ALTERNATIVAS AVALIADAS

6 PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO

6.1 PROCEDIMENTOS DE TESTE

6.2 CRITÉRIOS DE ACEITE E DESEMPENHO

7 ANÁLISE DOS RISCOS

8 CRONOGRAMA DO PROJETO

9 CONCLUSÃO

REFERÊNCIAS

MDPI. A survey on large language models in software security: Opportunities and threats. **Computers**, v. 15, n. 4, 2026. Disponível em: <https://www.mdpi.com/2073-431X/15/4/226>. Acesso em: 01 set. 2026.

State of Cyber. **AI-Assisted Code Introduces 4.4x More Security Vulnerabilities Than Human-Written Code, New Research Finds**. 2026. The Tennessean. Disponível em: <https://www.tennessean.com/press-release/story/223693/ai-assisted-code-introduces-4-4x-more-security-vulnerabilities-than-human-written-code-new-research-finds/>. Acesso em: 01 set. 2026.

TechScience. Static analysis techniques for secure software: A systematic review. **Journal of Cyber Security**, v. 7, n. 1, 2025. Disponível em: <https://www.techscience.com/JCS/v7n1/64045/html>. Acesso em: 01 set. 2026.

WANG, Y. *et al.* A generative ai cybersecurity risks mitigation model for code generation: Using ann-ism hybrid approach. **Scientific Reports**, 2025. Disponível em: <https://www.nature.com/articles/s41598-025-34350-3>. Acesso em: 01 set. 2026.

WU, Z. *et al.* **AI Code in the Wild: Measuring Security Risks and Mitigations**. 2025. ArXiv:2512.18567. Disponível em: <https://arxiv.org/abs/2512.18567>. Acesso em: 01 set. 2026.

YANG, X. *et al.* **ZeroFalse: Improving Precision in Static Analysis with LLMs**. 2025. ArXiv:2510.02534. Disponível em: <https://arxiv.org/abs/2510.02534>. Acesso em: 01 set. 2026.